МГТУ им. Н. Э. Баумана

Рубежный контроль №1

по курсу «Парадигмы и конструкции языков программирования»

Вариант запросов	E
Предметная область	№14: CD-диск, Библиотека CD-дисков
Автор	Козлов Максим
Группа	РТ5-31Б

1. Постановка задачи

Разработать программу на языке Python с двумя классами данных, связанными отношениями один-ко-многим и многие-ко-многим, создать списки объектов с тестовыми данными и реализовать запросы варианта E. Запросы сформулированы в условии для примера «Сотрудник и Отдел», поэтому термины перенесены на предметную область так:

В условии	В программе
Сотрудник	CD-диск (класс **CD**)
Отдел	Библиотека CD-дисков (класс **CDLibrary**)
Сотрудники отдела	Диски библиотеки (класс **CDInLibrary**)
Фамилия сотрудника	Название диска (поле **title**)
Зарплата	Цена диска в рублях (поле **price**)
Слово «отдел» в названии	Слово «библиотека» в названии

E1 (один-ко-многим). Вывести все библиотеки, в названии которых есть слово «библиотека», и список дисков в каждой.

E2 (один-ко-многим). Вывести библиотеки со средней ценой дисков, округлённой до 2 знаков после запятой, с сортировкой по средней цене.

E3 (многие-ко-многим). Вывести все диски, название которых начинается с буквы «А», и названия их библиотек.

2. Текст программы (rk1.py)

```
# используется для сортировки
from operator import itemgetter


class CD:
    """CD-диск"""

    def __init__(self, id, title, price, lib_id):
        self.id = id
        self.title = title
        self.price = price
        self.lib_id = lib_id


class CDLibrary:
    """Библиотека CD-дисков"""

    def __init__(self, id, name):
        self.id = id
        self.name = name


class CDInLibrary:
    """
    'Диски библиотеки' для реализации
    связи многие-ко-многим
    """

    def __init__(self, lib_id, cd_id):
        self.lib_id = lib_id
        self.cd_id = cd_id


# Библиотеки CD-дисков
libraries = [
    CDLibrary(1, 'Городская библиотека CD-дисков'),
    CDLibrary(2, 'Фонотека университета'),
    CDLibrary(3, 'Библиотека музыкальных записей'),
    CDLibrary(4, 'Домашняя библиотека коллекционера'),
]

# CD-диски
cds = [
    CD(1, 'Ария. Герой асфальта', 650, 1),
    CD(2, 'Кино. Группа крови', 725, 1),
    CD(3, 'Аквариум. Радио Африка', 590, 2),
    CD(4, 'ДДТ. Актриса весна', 480, 1),
    CD(5, 'Алиса. Энергия', 515, 3),
]

# Диски библиотек (связь многие-ко-многим)
cds_libraries = [
    CDInLibrary(1, 1),
    CDInLibrary(1, 2),
    CDInLibrary(1, 4),
    CDInLibrary(2, 3),
    CDInLibrary(3, 5),

    CDInLibrary(2, 1),
```

```
    CDInLibrary(4, 3),
    CDInLibrary(4, 5),
]


def main():
    """Основная функция"""

    # Соединение данных один-ко-многим
    one_to_many = [(cd.title, cd.price, lib.name)
                   for lib in libraries
                   for cd in cds
                   if cd.lib_id == lib.id]

    # Соединение данных многие-ко-многим
    many_to_many_temp = [(lib.name, link.lib_id, link.cd_id)
                         for lib in libraries
                         for link in cds_libraries
                         if lib.id == link.lib_id]

    many_to_many = [(cd.title, cd.price, lib_name)
                    for lib_name, lib_id, cd_id in many_to_many_temp
                    for cd in cds if cd.id == cd_id]

    print('Задание E1')
    # ключ - библиотека, значение - список названий дисков
    res_1 = {lib.name: [title for title, _, _ in
                        filter(lambda i: i[2] == lib.name, one_to_many)]
             for lib in libraries
             if 'библиотека' in lib.name.lower()}
    print(res_1)

    print('\nЗадание E2')
    res_2_unsorted = []
    # Перебираем все библиотеки
    for lib in libraries:
        # Цены дисков библиотеки
        lib_prices = [price for _, price, _ in
                      filter(lambda i: i[2] == lib.name, one_to_many)]
        # Если библиотека не пустая
        if len(lib_prices) > 0:
            # Средняя цена дисков библиотеки
            lib_avg = round(sum(lib_prices) / len(lib_prices), 2)
            res_2_unsorted.append((lib.name, lib_avg))

    # Сортировка по средней цене
    res_2 = sorted(res_2_unsorted, key=itemgetter(1), reverse=True)
    print(res_2)

    print('\nЗадание E3')
    # ключ - название диска, значение - список библиотек
    # 'A' здесь - кириллическая буква
    res_3 = {cd.title: [lib_name for _, _, lib_name in
                        filter(lambda i: i[0] == cd.title, many_to_many)]
             for cd in cds
             if cd.title.startswith('A')}
    print(res_3)


if __name__ == '__main__':
    main()
```

3. Результаты выполнения

```
Задание E1
{'Городская библиотека CD-дисков': ['Ария. Герой асфальта', 'Кино. Группа крови', 'ДДТ.
Актриса весна'], 'Библиотека музыкальных записей': ['Алиса. Энергия'], 'Домашняя
библиотека коллекционера': []}

Задание E2
[('Городская библиотека CD-дисков', 618.33), ('Фонотека университета', 590.0),
('Библиотека музыкальных записей', 515.0)]

Задание E3
{'Ария. Герой асфальта': ['Городская библиотека CD-дисков', 'Фонотека университета'],
'Аквариум. Радио Африка': ['Фонотека университета', 'Домашняя библиотека коллекционера'],
'Алиса. Энергия': ['Библиотека музыкальных записей', 'Домашняя библиотека коллекционера']}
```

Слово «библиотека» ищется без учёта регистра, поэтому в E1 попадает и «Библиотека музыкальных записей». «Фонотека университета» этого слова не содержит и в E1 не выводится.

У «Домашней библиотеки коллекционера» нет дисков по связи один-ко-многим: в E1 она выводится с пустым списком, а в E2 пропускается, так как среднее для неё не определено. По связи многие-ко-многим диски у неё есть, поэтому она присутствует в E3.